

Reading the COVID-19 Infection Global Map in R

Data visualisation tools provide an effective way to understand complex data. They help us identify the data growth trends, and visualise the outliers and patterns in data. Visualisation of data is done by using graphical elements like charts, graphs and maps. This article will help the reader to learn the visualisation techniques of R.



R supports a wide variety of graphical tools implemented with different packages. Though conventional graphical tools like line, plot, bar-plot and histogram are available within basic packages, high-end tools like ggplot, ggmmap and tidyverse require packages to be loaded with all of their dependencies.

Maps

The *ggmap* package is one of the important data visualisation tools. However, it requires several other essential data and attributes also. The data requires proper processing and arrangement to mould it into proper shape before mapping it on to the map object. Preprocessing performs filtering and data manipulation to make data compatible with map data.

Mapping also requires latitude and longitude information for each mapping object. Proper care is required to map the location information. Here, we shall try to learn the technique in steps.

ggplot

The *ggplot2* package, created by Hadley Wickham, is a useful graphics language for creating stylish and complex plots. Often the *qplot()* function is used to draw *ggplot* graphics to hide the complexity of the *ggplot* graphics syntax. It is a declarative system to create graphics and the implementation is based on 'The Grammar of Graphics' by Leland Wilkinson. This function maps variables to aesthetics, followed by which graphical primitives use them and take care of the details.

This package can be installed by installing the *ggplot2* package or simply from the whole tidyverse package. An alternative option is to install it from the development version of GitHub.

1. Install *ggplot* by installing the whole tidyverse package:

```
install.packages("tidyverse")
```

2. Alternatively, install only *ggplot2*:

```
install.packages("ggplot2")
```

3. Or from the development version of GitHub:

```
devtools::install_github("tidyverse/ggplot2")
```

Usage

As it is a complex graphical tool based on a deep-rooted theoretical platform, a detailed discussion of the functioning of *ggplot2* is not possible here. Here it will be illustrated with only two examples.

In most of the cases *ggplot()* starts with a data set and an aesthetic mapping with *aes()*. Then layers of different functionalities incorporate different graphical features and shapes into the graph. Graphical shapes are added with *geom.point()*, *geom.histogram()*, etc, and features are added with *scale_colour_brewer()*, *facet_wrap()*, *coord_flip()*, etc. Here are two examples of *ggplot* graphs on *mtcars* data.

Example 1:

```
library(ggplot2)
ggplot(mtcars, aes(mpg, hp, colour = cyl))
+ geom_point()
```

Example 2:

```
ggplot(mtcars, aes(mpg, colour = cyl)) +
geom_histogram()
```

Similarly, *geom_line()*, *geom_boxplot()*, *geom_bar()* will plot line, boxplot and bar graphs.

